

PRD

PRD — Intercity Ride Platform (Uganda)

1. Vision

A web-based marketplace connecting independent drivers and operators running long-distance journeys (Kampala–Mbarara, Soroti–Kampala, Jinja–Mbale, etc.) with passengers looking to travel those routes. Passengers search and book seats or a whole vehicle; drivers publish scheduled trips against reusable routes. The platform's revenue comes from a driver subscription, not from a cut of the fare.

2. Users & Roles

Role	Who
Passenger	Anyone searching for and booking a trip
Driver	Publishes trips, manages vehicles, views leads
Operator	Owns vehicles and/or employs drivers (individual, fleet, or company)
Admin	Verifies drivers/vehicles, manages routes and plans, handles disputes
Support agent	Handles tickets

A single person can hold more than one role (e.g. an individual driver is also their own operator) — enforced via `user_roles` , not a single column.

3. Core Concepts

- **Route** — a reusable path (Kampala → Jinja → Iganga → Mbale).
- **Trip** — one driver's scheduled instance of a route, with a vehicle, departure time, and seat capacity.
- **Booking** — a passenger's reservation against a trip (seat or whole vehicle).

- **Booking segment** — the actual capacity-holding unit: a booking may reserve seats only for part of the trip (Kampala→Jinja), leaving that capacity free for another passenger on a different segment (Jinja→Mbale) of the *same* vehicle. This is what makes multi-stop resale of seats possible. A private-vehicle booking is simply one segment spanning the whole trip.

4. V1 Features by Role

Passenger

- Phone OTP login
- Search by origin, destination, date, seats (matches trips whose stop list covers the requested origin → destination, not just exact route matches)
- "Alert me" when a search returns nothing
- View trip detail: driver, vehicle, stops, price, seats available
- Book a seat or a whole vehicle; pay the driver directly (cash or the driver's own mobile money number) — nothing is paid through the app
- E-ticket with QR/booking reference
- Cancel a booking
- Live tracking on trip day; share trip with an emergency contact; SOS
- Rate the driver after the trip

Driver

- Sign-up with document upload, held for admin verification
- Vehicle management
- Publish/edit trips against a route, set price within an optional band
- View and manage bookings on a trip
- Subscribe to a plan (trips/period + leads/period); pay by mobile money
- View and reveal "radar" leads (passengers who alerted on a route with no matching trip)
- Start/complete a trip, share live location
- Rate passengers

Admin

- Verify drivers, vehicles, documents
- Manage towns, pickup points, routes
- Manage subscription plans
- Live trip map, incident/SOS console
- Support ticket queue
- Demand analytics from `search_events` (which routes need more supply)

5. Payment & Subscription Model (V1)

The platform never touches fare money. Passengers pay drivers directly — cash on board or a manual mobile-money transfer to the driver's own number. The only thing the platform charges for is the driver's subscription:

- **Plans** cap two things per period: trips a driver can publish, and "leads" (passenger contacts) they can reveal from the radar.
- Drivers pay via MTN/Airtel mobile money through an aggregator (Flutterwave, Pesapal, or similar) at the start.
- Passengers are never limited or gated — every trip is visible to every passenger regardless of the driver's plan. Limiting driver-side benefits (radar visibility, trip volume) rather than what passengers can see is what keeps trust intact.
- Expiry never cancels a trip that already has bookings; a grace period applies before a driver drops to the free tier.

This removes the need for commission tracking, driver earnings ledgers, or payouts in V1 — `payments` exists only to record subscription and lead top-up charges.

6. Non-Functional Requirements

- **RLS everywhere.** Every table in `schema.sql` needs a row-level security policy before production; `driver_documents`, `payments`, and `driver_subscriptions` are the highest-sensitivity ones.
- **Search performance.** Index on `(route_id, departs_at)` for trips and `(trip_id, sequence)` for stops — see `schema.sql` for the indexes already defined.

- **Booking correctness.** All seat reservation goes through `book_segment()`, which serializes concurrent attempts on the same trip with an advisory lock — never insert into `booking_segments` directly from the app.
- **Low-bandwidth tolerance.** Build as an installable PWA; assume patchy connectivity outside Kampala.

7. Out of Scope for V1 (explicitly deferred)

- Named/assigned seat numbers (add a `trip_seats` table later only if an operator actually needs reserved seating)
- Platform-collected fares, commission, or driver payouts
- Passenger-requested private rides (Uber-style on-demand dispatch)
- Dynamic/algorithmic pricing
- In-app chat, promotions, wallet
- `promotions` / `promotion_redemptions` tables exist in the schema for future stability but are not built into any V1 flow

8. Database Schema Reference

Full definitions, comments, and functions are in `schema.sql`. Grouped by domain:

Identity

Table	Purpose
<code>profiles</code>	Public user record, one per authenticated user
<code>user_roles</code>	Which role(s) each user holds

Operators & fleet

Table	Purpose
<code>operators</code>	Owns vehicles/drivers — individual, fleet, or company
<code>driver_profiles</code>	License, verification status, rating for a driver
<code>driver_documents</code>	Uploaded license/ID/insurance docs, admin-reviewed
<code>vehicles</code>	A vehicle owned by an operator

Table	Purpose
vehicle_documents	Logbook/insurance/inspection uploads per vehicle

Geography

Table	Purpose
towns	Top-level place used in search and route origin/destination
pickup_points	A specific stage/terminal/landmark inside a town

Routes

Table	Purpose
routes	A reusable path between two towns
route_stops	The template stop order for a route

Trips

Table	Purpose
trips	One driver's scheduled journey on a route
trip_stops	The actual stop list for one trip

Bookings

Table	Purpose
bookings	A passenger's reservation
booking_segments	The capacity-holding unit; enables reselling freed seats between stops
booking_passengers	Named travelers under one booking

Discovery & radar

Table	Purpose
search_events	Every search, including empty ones — demand signal
trip_alerts	A passenger opts in to be notified/visible for an unmatched search
lead_views	Records a driver spending a lead credit on an alert

Subscriptions

Table	Purpose
subscription_plans	Price, period, trip/lead caps per tier
subscription_features	Non-numeric perk flags per plan
driver_subscriptions	A driver's subscription period, one row per renewal
subscription_usage	Per-period counters checked on every trip post / lead reveal

Payments

Table	Purpose
payments	Subscription/lead-topup charges only — never fares

Trust & safety

Table	Purpose
ratings	Passenger↔driver ratings
emergency_contacts	A user's trusted contacts
trip_shares	Shareable live-tracking link for a booking
incidents	SOS, accidents, breakdowns, harassment reports

Tracking & audit

Table	Purpose
trip_locations	Periodic GPS pings during an active trip
trip_events	State-change audit trail (published, started, delayed, completed...)

Communication

Table	Purpose
notifications	In-app/push/SMS notification log
push_notifications	Web push registration per user
support_tickets	Support requests
support_messages	Messages within a ticket

Marketing (post-V1)

Table	Purpose
promotions	Discount codes
promotion_redemptions	Usage of a discount code

9. Decisions Made While Reconciling the Two Designs

Flagging these explicitly since they resolve a disagreement or fill a gap between the original design and the colleague's outline — confirm before building:

1. **Capacity-based segments, not named seats.** Chosen because Ugandan shared-taxi vehicles rarely assign seat numbers. Reversible later by adding a `trip_seats` table and an optional `seat_id` on `booking_segments`.
2. **Trips are scheduled-only.** The colleague's outline included an `immediate` trip type; dropped since drivers post scheduled trips only.
3. **Two-level geography** (`towns` + `pickup_points`) replaces the colleague's `cities / locations` pair, which had unclear overlap.

4. **No commission/earnings/payout tables.** The colleague's outline had `driver_earnings` , `driver_payouts` , and `cash_collections` under the assumption the platform takes a cut of fares. Under the agreed subscription model the platform never handles fare money at all, so these were dropped in favor of `payments` covering subscriptions only.
5. `sos_events` merged into `incidents` via a `kind` column — same shape, same workflow.
6. **Seat holds with expiry** (`booking_segments.expires_at` , `expire_stale_holds()`) were missing from the colleague's outline and are required to prevent two passengers both passing an availability check during checkout.